Why does the CoffeeScript compiler do all this rewriting? Because rewriting simplifies the next compilation step.

8.3.3. The abstract syntax tree

Once the compiler has a token stream, it's ready to use it to create an abstract syntax tree (AST). Whereas the token stream represents the syntax of the program, the AST represents the meaning of the program in terms of the rules of the CoffeeScript grammar. Going back to the simple function expression for which Scruffy wants to provide an alternative syntax, what does the AST for that look like? To get the AST you invoke `coffee.nodes` with either a string of CoffeeScript or a CoffeeScript token stream:

```
coffee = require 'coffee-script'

expression = 'I = (x) -> x'

tokens = coffee.tokens expression

coffee.nodes tokens
# { expressions:
#   [ { variable: [Object],
#       value: [Object],
#       context: undefined,
#       param: undefined,
#       subpattern: undefined } ] }
```

The default string representation of an object on the REPL lacks some details important for understanding what's going on here. By using `JSON.stringify` and formatting the result, you can better see the AST:

```
console.log JSON.stringify coffee.nodes, null, 2
```

Now you see the AST object for the `I = (x) -> x` expression:

```
{
  "expressions": [
    {
      "variable": {
        "base": {
          "value": "I"
```